

FULL-STACK SOCIAL MEDIA APPLICATION

Mingle

A focused social experience for sharing posts, media, reactions, comments, and saved content.

What is Mingle?

Product idea

Mingle is a social network-style application where authenticated users can create and discover updates, interact with posts, and manage a personal profile.

Primary experience

The home feed is the center of the product: posts are loaded from the API, rendered as cards, and progressively fetched as the user scrolls.

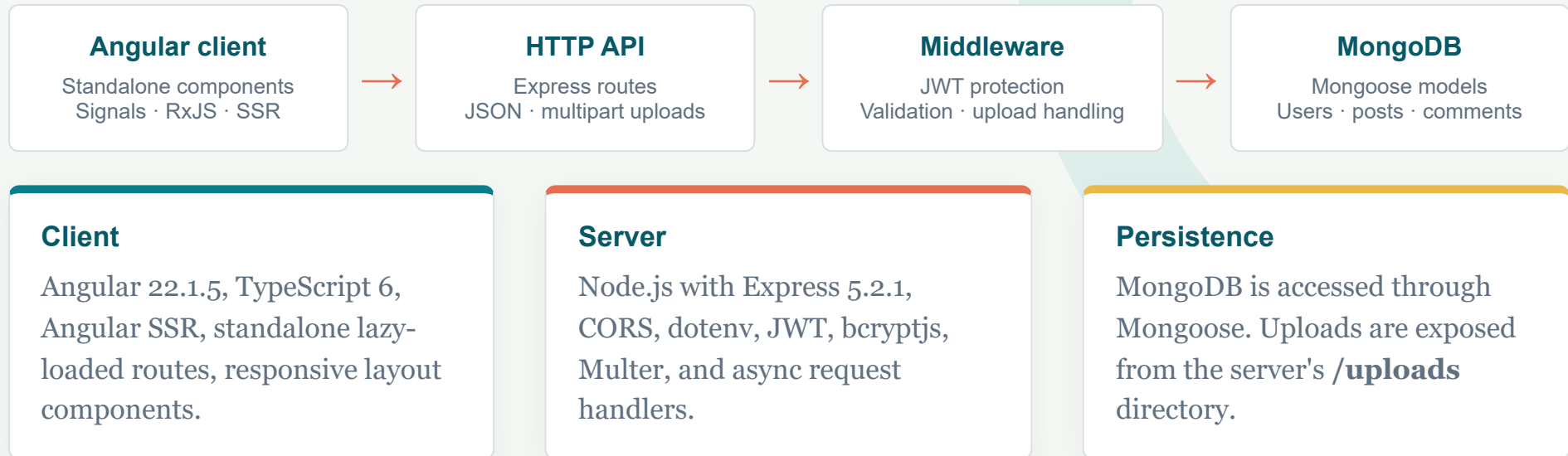
Audience

People who want a lightweight space for personal updates, visual media, conversation, and quick social actions.

Current product surface

Authentication, home feed, post creation, explore, profile, edit profile, saved posts, comments, responsive navigation, and reserved routes for future features.

Two applications, one API contract



Core user journeys

Join and sign in

- Registration with name, username, email, and password.
- Password hashing with bcryptjs.
- JWT login token with one-day expiry.
- Guest and authenticated route guards.

Share and discover

- Create posts with text and up to five media files.
- Image and video rendering with alt text.
- Home feed sorted newest first.
- Explore post and user search.

Interact and manage

- Like and unlike posts and comments.
- Save and unsave posts.
- Create, edit, delete, and like comments.
- View and update profile information.

Responsive navigation includes side navigation on larger screens and bottom navigation on smaller screens.

Infinite scroll with server pagination

Backend contract

```
GET /api/posts?page=1&limit=10
```

```
{ success: true,  
  data: {  
    data: [ ...posts ],  
    page: 1,  
    limit: 10,  
    count: 10  
  }  
}
```

Client behavior

- An IntersectionObserver watches a bottom sentinel.
- Requests are guarded so only one page loads at a time.
- New pages append to the existing signal.
- Duplicate post IDs are filtered.
- Loading stops when a page contains fewer than 10 posts or is empty.

Page 1

Initial feed request



Sentinel visible

Observer fires near bottom



Page N

Append posts to feed



End reached

Stop when page is short

Domain objects and relationships

User

Identity, email, hashed password, role, bio, avatar, deletion status, and references to saved posts.

name username email avatar

savedPosts[]

Post

Author, text content, media collection, visibility, likes, timestamps, and soft-delete fields.

author content media[] likes[]

visibility

Comment

Text, user, post, optional parent comment for nesting, likes, timestamps, and soft-delete fields.

content user post

parentComment likes[]

Relationship summary

A user authors posts, comments belong to both a user and a post, posts maintain a list of users who liked them, and users maintain saved-post references.

Server routes at a glance

Authentication · /api/auth

- **POST /register** creates an account.
- **POST /login** returns a JWT.
- **GET /protected** checks a legacy auth middleware path.

Users · /api/users

- **GET /me** retrieves the current user.
- **GET /:userId** retrieves a user.
- **PATCH /:userId** updates profile and avatar.
- **DELETE /:userId** removes a user.

Posts · /api/posts

- CRUD operations and paginated feed.
- Search and saved-post retrieval.
- Like/unlike and save/unsave actions.
- Multipart media upload support.

Comments · /api/comments

- Create and list comments by post.
- Update and delete owned comments.
- Like, unlike, and inspect comment likes.

Where the code lives

client/src/app

```
core/  
  auth-interceptor.ts  
  guards/  
  layout/  
features/  
  auth/  
  explore/  
  feed/  
  post/  
  profile/  
  static/  
shared/  
  components/  
  models/
```

server

```
controllers/  
  authController.js  
  commentController.js  
  postController.js  
  userController.js  
middlewares/  
  authMiddleware.js  
  uploadMiddleware.js  
  validationMiddleware.js  
model/  
  User.js · Post.js · Comment.js  
routes/  
  authRoutes.js · postRoutes.js  
  userRoutes.js · commentRoutes.js
```

The client uses lazy-loaded standalone components; the server separates routing, controllers, models, middleware, and utilities.

Local development workflow

Start the API

```
cd server  
npm install  
npm run dev
```

The server uses **PORT** from the environment, defaulting to 5000. The client currently targets localhost:3000 for API calls, so align the configured port before running end to end.

Start the client

```
cd client  
npm install  
npm start
```

Open **http://localhost:4200/**. Production build: **npm run build**. Unit tests: **npm test**.

Required environment

Configure the MongoDB connection and JWT secret in the server environment. The client and API must agree on the API host and port. Authenticated requests require the JWT interceptor/token flow.

What is complete, and what remains

Implemented

- Authenticated Angular shell and route guards.
- Post feed with infinite scroll pagination.
- Post creation with media upload.
- Likes, saves, comments, profile, and explore flows.
- Responsive desktop and mobile navigation.

Next improvements

- Add automated server tests and focused feed component tests.
- Return total pages or an explicit **hasMore** field from the feed API.
- Resolve the configured client/API port mismatch for local startup.
- Complete notifications, messages, and settings routes.
- Add production environment configuration and stronger error UX.

This summary is based on the current source tree and reflects the implementation available in August 2026.